

Wind-Deformable 3DGS

구현 현황 체크리스트

Wind3DGS

Created: 2026-05-06 Last updated: 2026-05-18

Development Pipeline

3DGS input → fixed-topology proxy → Gaussian binding → wind deformation →
Gaussian transport → SH residual correction → Gaussian rendering

이 문서는 구현 진행 상황을 관리하기 위한 status board다. 연구 방향 자체가 바뀌는 결정은 기존 규칙대로 ideas/idea_sketch.tex와 ideas/changelog.md에 기록한다.

상태 표기

상태	의미
☐	아직 시작하지 않음
[~]	진행 중
☒	완료
[!]	막힘 또는 결정 필요

실험 폴더 naming convention

앞으로 새 구현 실험은 시간순 expNNN prefix보다 development milestone tag를 우선 사용한다.

권장 형식

experiments/M##_short_name/

- ☐ experiments/M01_static_3dgs_io/
- ☐ experiments/M02_mesh_proxy_binding/
- ☐ experiments/M03_procedural_wind/

여기서 M##는 이 체크리스트의 milestone 번호와 대응된다. 파일 정렬이 편하도록 folder name에는 M1 대신 M01, M2 대신 M02처럼 두 자리 숫자를 쓴다.

기존 experiments/exp001_*, exp002_*, exp003_* 폴더는 초기 planned experiment slots로 보고, 명시적으로 정리하기 전까지는 유지한다.

코드와 실험 산출물 분리 원칙

- ☐ reusable implementation은 code/wind3dgs/ 아래 module로 관리한다.
- ☐ experiments/는 experiment README, generated assets, outputs, reports, 그리고 backward-compatible wrapper만 둔다.
- ☐ 기존 experiment-local 실행 명령은 wrapper로 유지한다.
- ☐ 새 개발은 code/wind3dgs/ module 경로를 기준으로 한다.

2026-05-18 Scope Control Update

전체 시스템은 너무 많은 구성 요소를 한 번에 핵심 기여처럼 보이지 않게 관리한다.

Core runtime stack

proxy deformation → Gaussian mean/covariance/frame transport → lightweight SH residual appearance correction → 3DGS rendering

- ☐ SH residual은 제거하지 않는다. 이것은 per-frame SH optimization 없이 runtime rendering을 가능하게 하는 필수 appearance correction module이다.
- ☐ 다만 논문 headline은 AI SH prediction이 아니라 runtime wind-deformable 3DGS representation으로 유지한다.
- ☐ controllable material fields, simulation anchors vs. load samples, gated floater likelihood, projection-risk-aware load sampling은 core를 보조하는 설계 장치다.
- ☐ MVP critical path에서는 simple fixed simulation anchors와 simple fixed material-space load samples부터 구현한다.
- ☐ view-masked gated floater와 projection-risk-aware load sampling은 robustness/refinement 단계로 두며, M4/M7/M8의 첫 성공 조건을 막지 않는다.
- ☐ advanced anchor/load scoring의 상세 기준은 ideas/anchor_method_revision_notes_2026-05-11_v3.tex를 reference로 유지한다.

현재 최우선 확인 질문

가장 먼저 확인해야 할 기술 질문은 다음과 같다.

proxy mesh가 변형될 때, 그 위에 bound된 Gaussian Splats가 position, orientation, covariance, rendering까지 자연스럽게 따라오는가?

여기에 runtime 질문을 하나 더 붙인다.

per-frame SH optimization 없이, transported Gaussian에 lightweight SH residual correction을 붙였을 때 rendering quality와 temporal stability가 유지되는가?

첫 질문을 먼저 검증한 뒤 3DGS-to-mesh extraction으로 넘어간다. 처음부터 mesh extraction을 붙이면, 결과가 깨졌을 때 원인이 extraction noise인지, Gaussian binding 문제인지, covariance transform 문제인지 구분하기 어렵다. 두 번째 질문은 geometry transport가 안정화된 뒤 반드시 M9에서 검증한다.

Milestone Overview

ID	Milestone	완료 기준	상태
M0	Project and Experiment Hygiene	첫 구현 실험 README에 goal, input, command, output, metrics, notes가 정리되어 있다.	☑
M1	Static 3DGS I/O Baseline	static rendering이 안정적으로 동작하고, 이후 deformed rendering의 reference로 사용할 수 있다.	[~]
M2	Synthetic Mesh-Proxy Binding MVP	mesh deformation에 따라 Gaussian motion이 안정적이며 popping, drift, covariance artifact가 없다.	[~]
M3	Procedural Wind Deformation	mesh-bound Gaussian asset을 wind parameters로 scriptable하게 animation할 수 있다.	☑
M4	Lightweight Mesh Simulator	proxy mesh가 단순 position warping이 아니라 connected topology와 constraints를 통해 wind에 반응한다.	☐

M5	Real 3DGS Asset to Proxy Binding	automatic mesh extraction 없이도 real 3DGS asset을 proxy mesh를 통해 animation할 수 있다.	☐
M6	3DGS-to-Mesh Proxy Extraction	적어도 하나의 extraction path가 stable binding과 wind simulation에 사용할 수 있는 proxy mesh를 만든다.	☐
M7	Wind Exposure From Gaussian Opacity	occluded 또는 shielded regions가 wind에 덜 반응하는 모습을 controllable하고 visually plausible하게 보여준다.	☐
M8	Force-Space Load Model	model이 stable simulation behavior를 유지하면서 wind response를 개선한다.	☐
M9	Runtime SH Residual Appearance Correction	learned residual correction이 canonical 또는 analytic-only appearance보다 quality gap을 줄이고, per-frame SH optimization 없이 runtime rendering path가 동작한다.	☐
M10	Evaluation and Paper Evidence	main paper claim을 지지할 수 있고 key ablations가 reproducible한 실험 결과가 있다.	☐

1 M0. Project and Experiment Hygiene

목표. 첫 구현 실험을 재현 가능하고 추적 가능한 형태로 시작한다.

- ☑ 첫 active implementation experiment 이름을 정한다.
 - 추천: `experiments/M02_mesh_proxy_binding/`
- ☑ 코딩 전에 experiment README를 만든다.
- ☑ 실행 명령어, 입력, 출력, 관찰 내용을 experiment README에 기록한다.
- ☑ 큰 generated outputs는 `code/outputs/`, `assets/`, 또는 experiment folder 아래에 둔다.
- ☑ reusable implementation code를 `code/wind3dgs/` package로 분리한다.
 - 현재 상태: M01/M02 Python 구현은 `code/wind3dgs/` 로 이동했고, `experiments/`에는 wrapper와 산출물이 남아 있다.

완료 기준. 첫 구현 실험 README에 goal, input, command, output, metrics, notes가 정리되어 있다.

2 M1. Static 3DGS I/O Baseline

목표. deformation을 넣기 전에 static 3DGS asset을 정상적으로 load하고 render할 수 있는지 확인한다.

- ☑ 작은 static 3DGS asset 하나를 선택하거나 준비한다.
 - 현재 상태: `experiments/M01_static_3dgs_io/assets/synthetic_leaf_3dgs.ply`를 생성했다.
 - asset은 leaf-like thin sheet 형태의 synthetic Inria-style 3DGS .ply다.
- ☑ Gaussian attributes loader를 구현하거나 기존 코드를 연결한다.
 - mean / position
 - scale
 - rotation
 - opacity
 - SH coefficients
 - 현재 상태: ASCII / binary_little_endian vertex-only .ply loader를 구현했다.
 - `scale_i`는 exp, `opacity`는 sigmoid, `rot_i`는 normalized quaternion으로 변환한다.
 - `f_dc`와 `f_rest`를 SH tensor로 구성하며 기본 render는 `sh_degree=0`을 사용한다.
- ☑ camera intrinsics와 extrinsics를 load한다.
 - 현재 상태: `cameras/synthetic_leaf_3dgs_cameras.json`에 K, `camera_to_world`, `view_matrix`를 저장하고 load한다.
 - convention은 world-to-camera, +z forward로 고정했다.
- ☑ canonical static views를 render한다.
 - 현재 상태: `canonical_front`, `canonical_oblique`, `canonical_side` PNG를 저장했다.
- ☑ 짧은 turntable 또는 fixed camera path 영상을 render한다.
 - 현재 상태: 48-frame turntable PNG와 `outputs/turntable.gif`를 저장했다.
- ☑ sanity statistics를 출력한다.
 - Gaussian count
 - bounding box
 - opacity range
 - scale range
 - SH coefficient shape

- 현재 상태: `outputs/stats.json`와 `outputs/render_report.md`에 기록한다.
 - ☑ baseline images 또는 video를 `experiment/output` folder에 저장한다.
 - 현재 상태: `outputs/` 아래에 canonical renders와 turntable preview를 저장했다.
 - 사용자 local shell에서 `--backend cpu_debug` 기준 canonical renders 3장과 turntable frames 48장 생성이 확인되었다.
 - [!] `gsplat` CUDA Gaussian rasterizer로 static rendering을 검증한다.
 - 현재 상태: `render_static_baseline.py`에 `auto`, `gsplat`, `cpu_debug` backend를 구현했다.
 - 1차 blocker였던 CUDA Toolkit / `nvcc` 누락은 사용자 local shell에서 해결했다.
 - 확인 결과: local GPU는 NVIDIA GeForce GTX 1080 Ti, Compute Capability 6.1이다.
 - 현재 blocker: `gsplat 1.5.3` JIT build가 `compute_61` target에서 CUDA cooperative groups compile error로 실패한다.
 - 원인: NVIDIA CUDA docs 기준 `labeled_partition`은 Compute Capability 7.0 이상이 필요하다. GTX 1080 Ti는 CC 6.1이라 현재 `gsplat 1.5.3` CUDA backend와 맞지 않는다.
 - 다음 액션: 이 machine에서는 M1을 `cpu_debug` fallback으로 유지하고, 논문용 `gsplat` render 검증은 CC ≥ 7.0 GPU에서 진행한다.
 - ground-truth views가 있으면 PSNR, SSIM, LPIPS를 계산한다.
 - 현재 상태: `synthetic/minimal` asset에는 ground-truth image가 없어 보류한다.
- 완료 기준.** `synthetic/minimal` static pipeline은 현재 machine에서 `cpu_debug` 기준으로 재현 가능하다. 다만 GTX 1080 Ti에서는 `gsplat 1.5.3` CUDA backend가 hardware capability 때문에 막혀 있으므로, 논문 품질의 static rendering reference는 CC ≥ 7.0 GPU에서 추가 확인해야 한다.

3 M2. Synthetic Mesh-Proxy Binding MVP

목표. 깨끗한 synthetic mesh deformation을 기준으로 Gaussian Splats가 제대로 따라 움직이는지 검증한다.

mesh extraction을 뒤로 미루는 이유

이 단계에서는 일부러 `3DGS-to-mesh extraction`을 하지 않는다. 먼저 깨끗한 synthetic mesh를 사용해서 Gaussian binding과 transform bug를 분리해서 본다.

- ☑ 간단한 proxy mesh를 생성하거나 load한다.
 - flag plane
 - cloth strip
 - leaf-like thin sheet
- ☑ mesh surface 위 또는 근처에 synthetic Gaussians를 sample한다.
- ☑ M1 synthetic Inria-style PLY를 M2 viewer에서 load하고 generated proxy mesh에 bind한다.
 - 현재 상태: `viewer_gpu.py`에 `--ply` 로드 경로를 추가했다.
 - 예시 asset: `experiments/M01_static_3dgs_io/assets/synthetic_leaf_3dgs.ply`
 - 현재 기본 방식: PLY Gaussian mean을 proxy grid에 rasterize하고, Gaussian이 존재하는 cell만 남기는 occupancy-extracted proxy mesh를 생성한다.
 - `--ply-proxy-mode bbox`를 사용하면 이전 rectangular bounding-box proxy로 비교할 수 있다.
 - 각 Gaussian은 proxy triangle의 `triangle_id`, `barycentric coordinates`, `local offset`으로 bind된다.
 - PLY의 `rot_0..3` quaternion은 anisotropic frame으로 변환한 뒤 triangle-local basis를 통해 transport한다.
- ☑ 각 Gaussian을 mesh triangle에 bind한다.

- triangle id
- barycentric coordinates
- triangle local frame
- local offset

☑ 단순하고 원인을 알 수 있는 mesh deformations를 적용한다.

- bending
- sine-wave flutter
- twist
- edge flap
- compound
- 현재 상태: `viewer_gpu.py`와 `viewer.html` 모두 UI에서 `sine`, `bend`, `twist`, `edge_flap`, `compound`를 선택할 수 있다.

☑ deformed triangle의 barycentric position으로 Gaussian centers를 update한다.

☑ triangle-local rotation으로 Gaussian covariance 또는 anisotropic frame을 transport한다.

- 현재 상태: canonical anisotropic frame coefficients를 triangle-local coordinates로 보존하고, deformed triangle basis에서 frame과 covariance를 재구성한다.
- `viewer_gpu.py`: sampled transported GS ellipsoid surface를 기본으로 표시하고, 필요하면 frame axes도 표시한다.
- `viewer.html`: anisotropic ellipse와 frame axes를 표시한다.

[~] frame flip, seam popping, exploding covariance, surface sliding이 생기는지 확인한다.

- 현재 상태: visual inspection과 numeric binding/covariance verification은 통과. `10x10`, `30x30`, `50x50` asset에 대해 `sine`, `bend`, `twist`, `edge_flap`, `compound` deformation을 모두 검증했다.
- 실제 3DGS rasterization과 SH appearance artifact 검증은 아직 전이다.

☑ dense visual inspection을 위한 GPU-backed debug viewer를 추가한다.

- 현재 상태: `viewer_gpu.py`는 ModernGL + GLFW 기반이며, `10x10`, `30x30`, `50x50` smoke test를 통과했다.
- `tkinter` control panel로 web viewer의 asset selector, mesh statistics, display toggles, deformation sliders, reset view, loaded file path UI를 제공한다.
- per-frame vertex normal shading과 distance-based depth cueing을 추가해 cloth 굴곡과 앞뒤 depth 구분을 개선했다.
- face surface는 opaque로 표시한다.
- back face culling은 기본 on이며, camera side에 따라 front-face winding을 ccw/cw로 보정한다. UI toggle / C key / `--no-backface-culling`으로 끌 수 있다.
- camera rotation은 yaw/pitch/roll 3축을 지원한다. left drag는 yaw/pitch, right drag 또는 Shift+left drag는 roll, UI slider와 arrow/Z/X key도 지원한다.
- transported GS ellipsoid surface는 UI toggle / E key / `--no-gaussian-ellipsoids`로 켜고 끌 수 있다.
- transported GS anisotropic frame axes는 UI toggle / O key / `--show-gaussian-frames`로 켜고 끌 수 있다.
- `--ply` CLI와 Tk control panel의 load PLY... /use sample GS 버튼으로 M1 synthetic PLY와 built-in sample GS를 전환할 수 있다.
- PLY mode는 `--ply-proxy-mode occupancy`를 기본으로 사용해 leaf 영역 밖 rectangular mesh를 줄인다.

[~] before/after frames 또는 짧은 animation을 render한다.

- 현재 상태: `viewer.html`은 zero-setup fallback이고, `viewer_gpu.py`는 dense sample Gaussian inspection 용도다. 실제 3DGS rendering은 아직 전이다.

완료 기준. mesh deformation에 따라 Gaussian motion이 안정적이며, 눈에 띄는 popping, drift, covariance artifact가 없다.

4 M3. Procedural Wind Deformation

목표. manual deformation을 단순한 wind-driven procedural deformation으로 바꾼다.

- ☒ lightweight wind field를 정의한다.
 - global direction
 - strength
 - gust frequency
 - spatial phase
 - 현재 상태: `code/wind3dgs/m03_procedural_wind/wind_field.py`에 one-way `WindParameters`와 procedural gust deformation을 추가했다.
 - M3 wind는 global wind direction + procedural spatial gust 방식이며, voxel wind grid나 two-way coupling은 아직 사용하지 않는다.
- ☒ synthetic mesh에 wind-like deformation을 적용한다.
 - 현재 상태: `viewer_gpu.py`의 `--deformation wind`로 M02 cloth proxy와 M1 synthetic leaf `PLY` occupancy proxy에 같은 wind deformation path를 적용할 수 있다.
 - anchored boundary는 고정되고, free vertices만 wind strength, gust frequency, spatial scale, turbulence에 반응한다.
- ☒ artist-style controls를 지원한다.
 - wind direction
 - amplitude
 - frequency
 - anchored boundary
 - 현재 상태: Tk control panel과 CLI에 `wind direction`, `wind spatial scale`, `turbulence`를 추가했다.
 - 기존 `amplitude`는 wind strength, `frequency`는 gust frequency로 사용한다.
- ☒ 다음 조건을 비교한다.
 - position-only Gaussian transport
 - position plus covariance transport
 - optional local-frame appearance correction
 - 현재 상태: `--transport-mode full`은 Gaussian center와 anisotropic frame/covariance를 함께 transport한다.
 - 현재 상태: `--transport-mode position_only`는 center만 mesh에 bind하고 anisotropic frame은 canonical frame으로 유지해 비교 baseline으로 쓴다.
 - local-frame / SH residual appearance correction은 M9의 core runtime module로 남긴다. M3에서는 geometry transport path만 검증한다.
- ☒ 여러 wind settings에 대한 qualitative videos를 저장한다.
 - 현재 상태: `code/wind3dgs/m03_procedural_wind/render_wind_preview.py`를 추가했다.
 - OpenGL viewer 없이 proxy mesh와 bound Gaussian centers를 orthographic CPU preview로 렌더링해 GIF, poster PNG, JSON report를 저장한다.
 - `calm`, `crosswind`, `gusty` preset을 50x50 cloth sample GS와 M1 synthetic leaf `PLY` occupancy proxy에 대해 생성했다.

완료 기준. mesh-bound Gaussian asset을 wind parameters로 scriptable하게 animation할 수 있다. 현재 module command와 experiment wrapper smoke test 기준으로 scriptable wind animation path가 동작하고, headless qualitative GIF도 생성된다.

5 M4. Lightweight Mesh Simulator

목표. procedural deformation에서 topology-aware simulator로 넘어간다.

- ☐ 작은 mass-spring, cloth, 또는 XPBD-style solver를 구현하거나 연결한다.
- ☐ persistent simulation anchors를 정의한다.
 - 기본 MVP: fixed proxy mesh vertices를 simulation anchors로 사용한다.
 - optional: 나중에 reduced simulation control nodes를 별도로 sample한다.
- ☐ anchored vertices 또는 boundary constraints를 추가한다.
- ☐ stiffness, damping, mass parameters를 추가한다.
- ☐ relative velocity와 surface normal을 사용한 wind force를 추가한다.
- ☐ simple fixed material-space load samples를 정의한다.
 - MVP에서는 uniform face samples 또는 face centers부터 시작한다.
 - **projection-risk-aware** refinement는 M8 이후 optional로 둔다.
- ☐ flag/cloth-strip test cases에서 안정적으로 simulation을 돌린다.
- ☐ Gaussian transport를 위해 mesh states over time을 export한다.

완료 기준. proxy mesh가 단순 position warping이 아니라 connected topology와 constraints를 통해 wind에 반응한다.

6 M5. Real 3DGS Asset to Proxy Binding

목표. binding pipeline을 real 또는 pretrained 3DGS assets에 연결한다.

- ☐ M1 loader를 통해 real/pretrained 3DGS asset을 load한다.
- ☐ low-opacity floaters를 제거하거나 mask한다.
- ☐ 필요하면 surface-oriented Gaussian attributes를 추정한다.
- ☐ 처음에는 manually provided proxy mesh 또는 simple proxy mesh로 시작한다.
- ☐ real Gaussians를 proxy mesh에 bind한다.
- ☐ real asset에 대해 M2/M3 deformation path를 실행한다.
- ☐ static result와 animated result를 render한다.

완료 기준. automatic mesh extraction 없이도 real 3DGS asset을 proxy mesh를 통해 animation할 수 있다.

7 M6. 3DGS-to-Mesh Proxy Extraction

목표. manually provided proxy mesh를 extracted simulation proxy로 대체한다.

- ☐ candidate extraction paths를 테스트한다.
 - SuGaR-style mesh extraction
 - Gaussian Opacity Fields-style extraction
 - TSDF fusion from rendered depths
 - point-cloud Poisson reconstruction
- ☐ extracted geometry를 simulation용으로 simplify 또는 remesh한다.
- ☐ proxy quality를 평가한다.
 - topology stability
 - triangle quality

- thin-structure preservation
 - reliability-weighted Gaussian transport demand
 - fixed-topology simulation anchorability
 - optional robustness score를 추가한다.
 - raw Gaussian count만 proxy 중요도로 쓰지 않는다.
 - opacity, surface distance, optional view evidence를 사용해 R_G 를 계산한다.
 - direct isolation reliability 대신 **view-masked gated floater likelihood**를 사용한다.
 - 이 항목은 M6 첫 extraction success의 blocker가 아니라 v3-based refinement다.
 - render-quality mesh extraction과 simulation-oriented proxy extraction을 비교한다.
- 완료 기준.** 적어도 하나의 extraction path가 stable binding과 wind simulation에 사용할 수 있는 proxy mesh를 만든다.

8 M7. Wind Exposure From Gaussian Opacity

목표. object의 어느 부분이 wind에 노출되거나 가려지는지 추정한다.

- 현재 Gaussian state를 wind axis 방향으로 rasterize한다.
- approximate transmittance 또는 exposure map을 계산한다.
- fixed material-space load sample에서 exposure를 sample한다.
 - 논문 notation: $q_\ell := (\tau_\ell, \mathbf{b}_\ell)$
 - τ_ℓ : proxy face/triangle index
 - $\mathbf{b}_\ell$: 해당 face 위 barycentric weights
 - frame마다 anchor를 다시 고르지 않는다.
 - load sample identity는 preprocessing 이후 고정한다.
- load-sample force를 barycentric accumulation으로 persistent simulation anchors에 project한다.
- exposure를 사용해 wind force를 modulate한다.
- with exposure와 without exposure animation을 비교한다.

완료 기준. occluded 또는 shielded regions가 wind에 덜 반응하는 모습을 controllable하고 visually plausible하게 보여준다.

9 M8. Force-Space Load Model

목표. physical integration을 명시적으로 유지하면서 aerodynamic loads를 learn 또는 calibrate한다.

- 각 load sample과 simulation anchor의 local state features를 정의한다.
- direct deformation이 아니라 forces 또는 torques를 predict한다.
- analytic 또는 parameterized load model을 먼저 구현한다.
- dense simulation labels가 있으면 dense forces를 sparse proxy anchors로 project한다.
 - 이 항목은 core MVP blocker가 아니라 supervised refinement다.
- net force와 wrench에 대한 conservation checks를 추가한다.
- optional: dense-force correction ratio로 **projection-risk-aware** load-sample refinement를 계산한다.
- minimal MLP 또는 learned residual load model을 train한다.
- analytic-only loads와 learned residual loads를 비교한다.

완료 기준. model이 stable simulation behavior를 유지하면서 wind response를 개선한다.

10 M9. Runtime SH Residual Appearance Correction

목표. expensive per-frame SH optimization 없이 deformation 이후 view-dependent appearance를 보존한다.
이 milestone은 optional polish가 아니라 runtime system의 필수 appearance stage다. 단, 논문 headline은 **AI SH prediction**이 아니라 proxy deformation, Gaussian transport, SH residual runtime correction의 결합으로 유지한다.

- ☐ appearance baselines를 세운다.
 - canonical SH only
 - analytic local-frame correction
 - per-frame SH optimization upper bound
 - learned SH residual
- ☐ residual labels 또는 pseudo-labels를 정의한다.
- ☐ local deformation과 wind features를 사용해 lightweight residual predictor를 train한다.
 - 입력 후보: local frame rotation, normal change, stretch, curvature change, wind direction/strength, material latent, canonical SH, binding features
- ☐ runtime inference path에 residual predictor를 연결한다.
- ☐ per-frame SH optimization 없이 animation sequence를 render한다.
- ☐ rendering quality와 temporal flicker를 평가한다.

완료 기준. learned residual correction이 canonical 또는 analytic-only appearance보다 의미 있는 quality gap을 줄이고, per-frame SH optimization 없이 runtime rendering path가 동작한다.

11 M10. Evaluation and Paper Evidence

목표. 구현 결과를 paper claim을 뒷받침하는 evidence로 만든다.

- ☐ synthetic scenes를 준비한다.
 - flag
 - cloth strip
 - curtain
 - leaf or plant-like sheet
- ☐ 가능하면 real 3DGS scenes를 준비한다.
- ☐ main baselines를 실행한다.
 - static 3DGS
 - direct Gaussian deformation
 - mesh-proxy deformation
 - with and without exposure
 - with and without SH residual
 - canonical SH vs analytic SH correction vs learned SH residual
- ☐ metrics를 측정한다.
 - PSNR / SSIM / LPIPS
 - temporal consistency
 - trajectory or surface displacement error
 - runtime / FPS
 - memory
- ☐ qualitative videos와 failure cases를 저장한다.

- 연구 방향이 바뀌면 `ideas/idea_sketch.tex`와 `ideas/changelog.md`를 업데이트한다.
완료 기준. main paper claim을 지지할 수 있고 key ablations가 reproducible한 실험 결과가 있다.

Open Decisions

- ☑ base로 사용할 3DGS implementation 또는 renderer는 무엇인가?
 - 결정: M1/M5의 우선 renderer는 `gsplat` CUDA Gaussian rasterizer로 두고, 개발 중에는 `cpu_debug` fallback을 유지한다.
- ☑ M1의 첫 static asset은 무엇으로 할 것인가?
 - 결정: real/pretrained asset 확보 전까지 `M01_static_3dgs_io`의 synthetic leaf-like Inria-style 3DGS asset을 사용한다.
- ☑ M2는 custom minimal renderer, existing 3DGS renderer, 또는 둘 다 사용할 것인가?
 - 결정: M02는 custom minimal GPU debug renderer를 사용하고, `viewer.html`은 fallback으로 유지한다. existing 3DGS renderer 연결은 M1/M5 이후로 미룬다.
- ☑ SH residual을 scope에서 뺄 것인가?
 - 결정: 빼지 않는다. SH residual은 runtime에서 per-frame SH optimization을 피하기 위한 필수 appearance correction module이다.
 - 단, headline contribution은 AI SH prediction이 아니라 runtime wind-deformable 3DGS representation으로 유지한다.
- ☑ advanced anchor/load scoring을 MVP critical path에 넣을 것인가?
 - 결정: 넣지 않는다. MVP는 fixed simulation anchors와 simple fixed material-space load samples부터 시작한다.
 - view-masked gated floater와 projection-risk-aware load sampling은 robustness/refinement 단계로 둔다.
- M4에서 사용할 mesh simulation library는 무엇인가?
- M6에서 가장 먼저 시도할 mesh extraction baseline은 무엇인가?
- 기존 `experiments/exp001_*-exp003_*` planned folders를 그대로 둘지, 나중에 milestone tag 기반으로 정리할지 결정한다.

Notes

- SH residual prediction은 M2/M3의 첫 geometry 목표는 아니지만, 최종 runtime system에서는 필수 module이다. M9에서 반드시 구현하고 M10 evidence에 포함한다.
- automatic mesh extraction부터 시작하지 않는다. 먼저 Gaussian-to-mesh binding과 deformation transport가 sound한지 증명한다.
- advanced anchor/load scoring부터 시작하지 않는다. fixed simulation anchors와 simple fixed load samples로 먼저 solver와 rendering loop를 닫는다.
- 각 milestone은 다음 layer를 붙이기 전에 runnable하고 visually inspectable해야 한다.
- M02의 GPU viewer는 구현 편의상 `ModernGL + GLFW`를 사용하지만, 연구 기여가 OpenGL 자체에 의존한다는 뜻은 아니다.